

Supplementary Materials for “Scalable Enumeration of Pareto-optimal Polymers for Computing Equilibrium Concentrations”

Archit Patil ✉ 

University of Texas at Austin, USA

Minki Hhan ✉ 

Korea Advanced Institute of Science and Technology, South Korea

David Soloveichik ✉ 

University of Texas at Austin, USA

1 Implementation Details

1.1 Covering Design Construction

We obtain the covering design given input parameters (n, k, t) as follows: First, when the parameters are sufficiently small ($n < 100$, $k \leq 25$, $t \leq 8$), we fetch the near-optimal covering designs from the precomputed covering design database, the La Jolla Covering Repository [1].

When the parameters are outside this range, we construct the covering design by combining smaller covering designs using dynamic programming [2]. This approach produces reasonably small covering designs, although we cannot ensure their near-optimality. This step can be done as preprocessing independent of the target system.

For the applications in this paper, the covering designs from the La Jolla repository with $k \leq 25$ are comparably efficient to those with $k > 25$ constructed by dynamic programming, as observed in Figure 1b of the supplementary materials.

1.2 Selecting the Optimal Block Size k

Algorithm 1 Covering Design Strategy

Require: Domain-monomer system (Σ, M) with $|M| = n$, parameters t and k with $t \leq k \leq n$

Ensure: $\hat{P}_{k,t}$

- 1: $\hat{P} \leftarrow \emptyset$
 - 2: Fetch or construct (n, k, t) -covering design $\mathcal{D}_{n,k,t}$
 - 3: **for** each block $B \in \mathcal{D}_{n,k,t}$ **do**
 - 4: Construct subsystem (Σ, B)
 - 5: Compute Hilbert basis H_B using `Normaliz`
 - 6: $\hat{P} \leftarrow \hat{P} \cup (\pi(H_B) \setminus \{0\})$
 - 7: **return** $\hat{P}$
-

Algorithm 1 requires the user to specify two parameters: the support bound t and the block size k . The user may choose t depending on the system to be analyzed and its known properties, and increasing the parameter t yields a reduced candidate set $\hat{P}_{k,t}$ or P_t^* with higher expressive power, at the cost of efficiency.

On the other hand, the choice of k is less obvious. For a fixed t , increasing k reduces the number of blocks in the $(|M|, k, t)$ -covering design but makes each Hilbert basis computation more expensive. The total runtime is therefore a non-monotone function of k .

We address this with a *probing* strategy. For each k and the $(|M|, k, t)$ -design $\mathcal{D}_k$, we can estimate the expected running time of a single Hilbert basis computation for block size k by

running a 100-iteration probe.

$$\hat{T}(k) = |\mathcal{D}_k| \times (\text{estimated run time of a single Hilbert basis computation}),$$

Starting from $k = t$, we estimate the total runtime for increasing values of k . We continue increasing k until the estimated runtime has increased for three consecutive values of k , and then select the value of k with the smallest estimated runtime observed.

This stopping rule is motivated by the tradeoff between the size of the covering design and the cost of each Hilbert basis computation. Increasing k reduces the number of blocks in the covering design, but it also makes each `Normaliz` call more expensive. So, the total runtime is expected to have an intermediate minimum. As shown in Figures 1a and 1b, once the estimated runtime passes this minimum, it increases monotonically in our tested systems. The probing step introduces only a negligible additive runtime overhead in our applications.

Figures 1a and 1b illustrate the runtime as a function of k for the Scaffolded DNA and AND gate linear cascade systems (Section 7.1 of the main text), respectively. As observed, the estimated runtime is very close to the actual runtime, and there is an optimal value of k that minimizes the runtime.

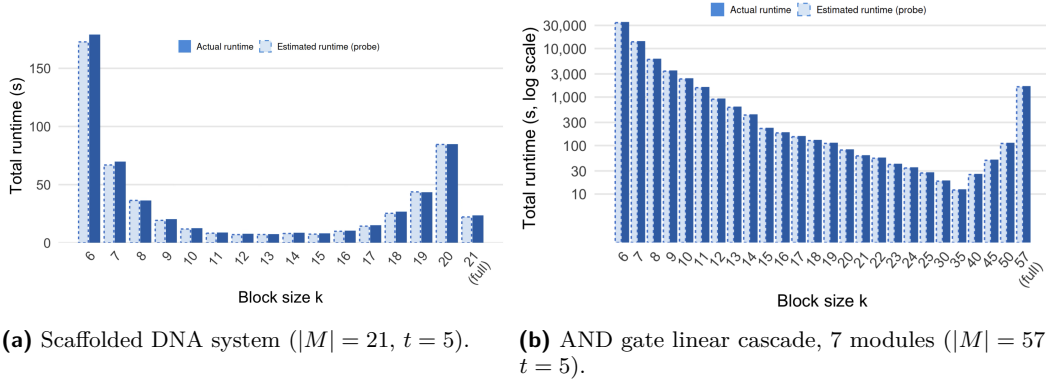


Figure 1 Probe-estimated total Hilbert basis runtime as a function of block size k , with support bound fixed at $t = 5$. Each point is the product of the per-block trial time (100-iteration probe) and the number of blocks in the covering design.

2 Leakage Analysis Plots

Section 7.4 of the main text analyzes leakage depending on which module has its input removed in a fixed-length AND gate linear cascade system. Figure 2 shows the final output concentration variation for this experiment across different values of E and concentration regimes A and B.

3 Equilibrium-Accuracy Plots

In Section 7.3 of the main text, we report the Regime B leakage condition with bond energy $E = -20$ kcal/mol. Here we include the corresponding concentration-error plots for both concentration regimes (A and B), each tested with different bond energies ($E = -20, -10$ kcal/mol). As shown in Figure 3, $t = 5$ provides a slightly more accurate equilibrium analysis compared to $t = 3$, although the concentrations of most equilibrium-relevant polymers are accurately captured even with the smaller support bound. Runtime for different cascade

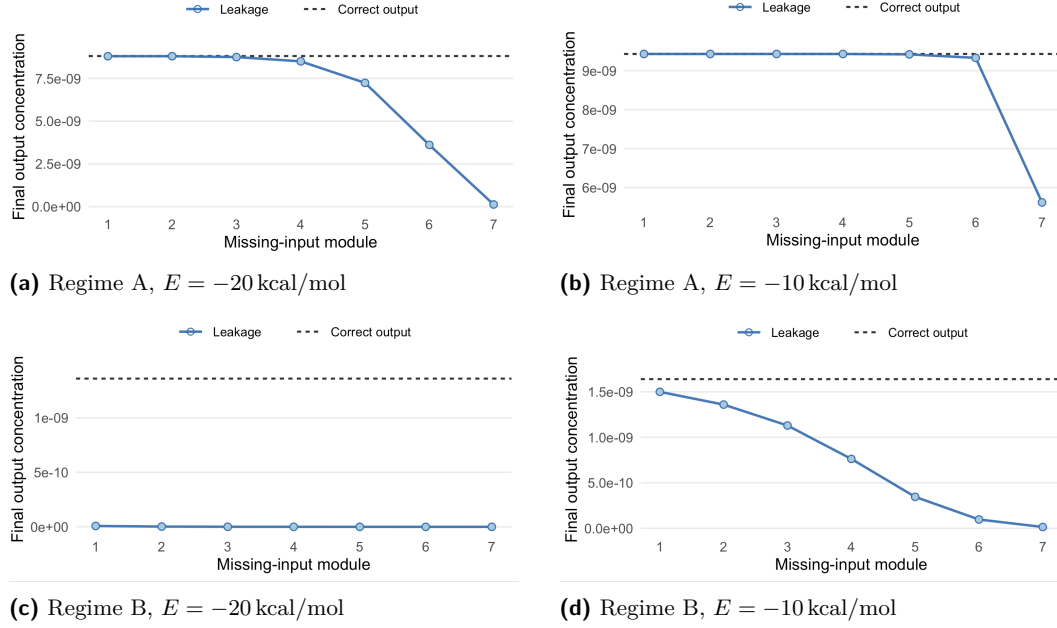


Figure 2 Final output concentration under single-input deletion in a seven-module linear cascade. The x -axis indicates the module whose input is removed. The solid curve shows the leakage concentration, and the dashed horizontal line shows the correct-output concentration when all inputs are present. The four panels compare Regimes A and B under bond energies $E = -20, -10$ kcal/mol.

lengths and values of t is compared in Figure 4. Although the runtime for the covering-design algorithm still scales exponentially with the number of modules, the rate of growth is much lower than that of the full Hilbert basis computation.

4 Domain Support Mode

This section introduces an alternative way to make the computation of the Pareto-optimal polymers scalable, based on the domain supports instead of the monomer supports. For a monomer m , we define the domain-support of m , denoted by $\text{supp}_\Sigma(m)$, as the set of domain-pair coordinates $d \in D$ such that the encoded vector $\mathbf{a}_m \in \mathbb{Z}^D$ has a nonzero entry at the d -th position, i.e., $\mathbf{a}_m(d) \neq 0$.

In the *domain mode*, we compute the Pareto-optimal polymers whose monomers are, for some t -element subset $T \subseteq D$, all in a set

$$M_T = \{m \in M \mid \text{supp}_\Sigma(m) \subseteq T\},$$

i.e., those monomers whose domains all lie within T . Similar to the monomer support case, we compute the set

$$\hat{P}_{k,t}^{\text{dom}} = \bigcup_{B \in \mathcal{D}} (\pi(H_B^{\text{dom}}) \setminus \{0\}),$$

where $\mathcal{D}$ is the $(|D|, k, t)$ -covering design and H_B^{dom} is the Hilbert basis for the augmented domain-monomer matrix $A'_B \in \mathbb{Z}^{|B| \times (|M_B| + 2|B|)}$ obtained by removing the irrelevant coordinates.

We omit the detailed analysis as it is analogous to the monomer mode.

Domain mode may be advantageous when the number of domains is relatively smaller than the number of monomers, or when many monomers share the same domain pairs, as in cascade and binary-tree TBNs, in which case the relevant covering design size $c(|D|, k, t)$ may be significantly smaller than $c(|M|, k, t)$.

References

- 1 Daniel Gordon. La jolla coverings repository, April 2026. doi:10.5281/zenodo.19735294.
- 2 Daniel M Gordon, Oren Patashnik, and Greg Kuperberg. New constructions for covering designs. *Journal of Combinatorial Designs*, 3(4):269–284, 1995.

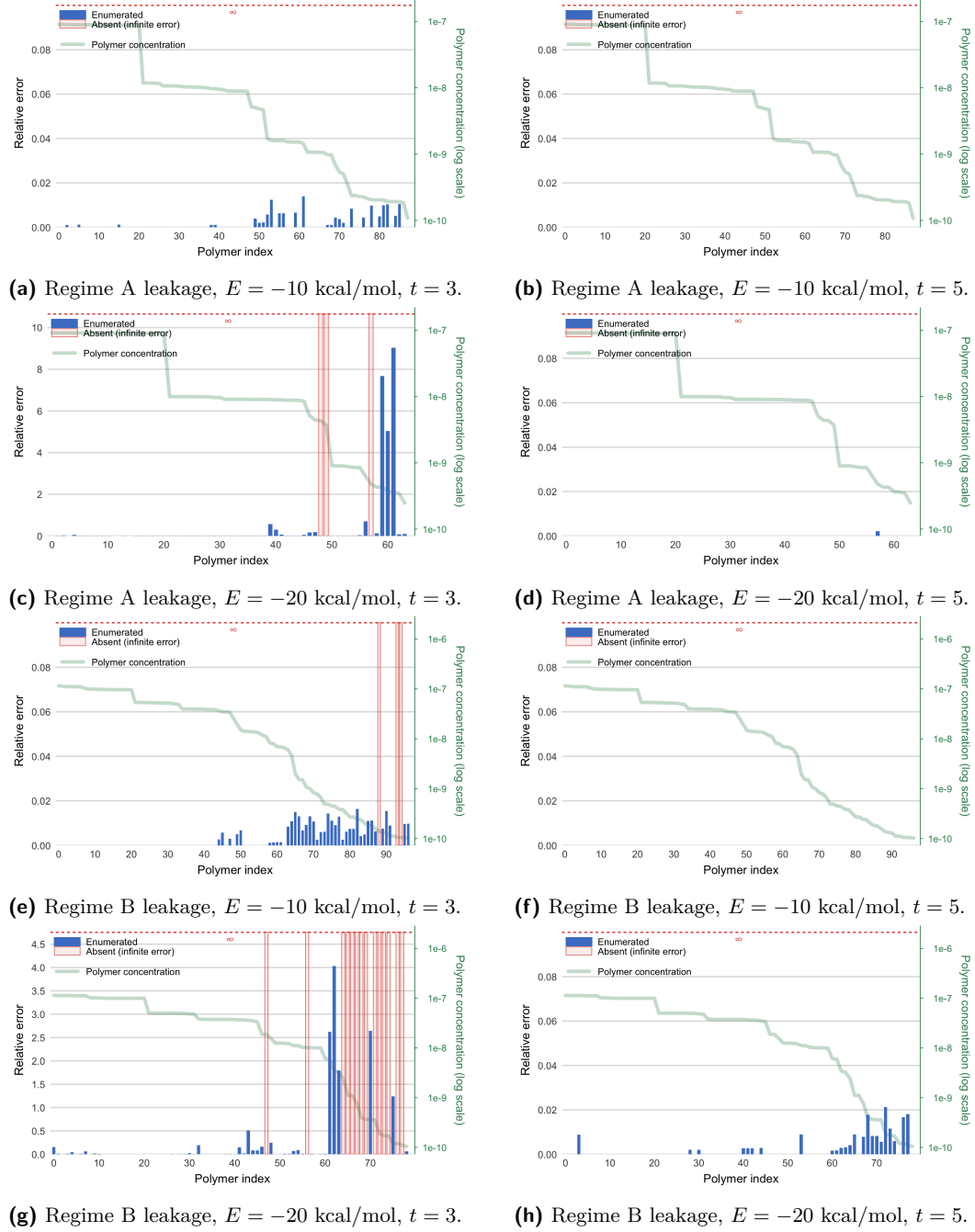


Figure 3 Relative concentration error for equilibrium-relevant polymers in the seven-module linear cascade under Regimes A and B with bond energies $E = -20, -10$ kcal/mol, mirroring saturated and unsaturated systems, respectively.

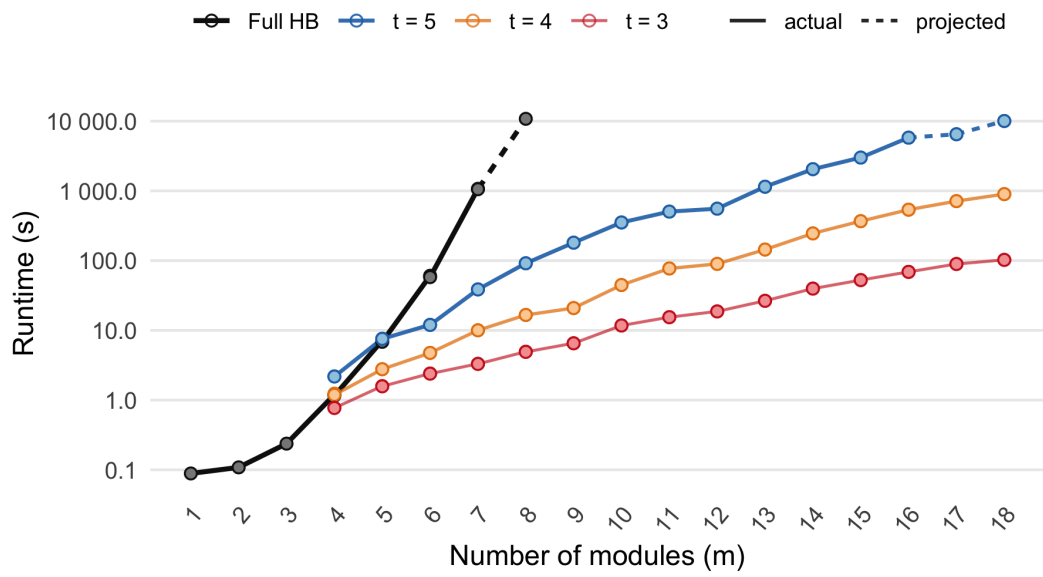


Figure 4 Runtime comparison for leakage experiments across linear cascade lengths and different parameters of the covering-design algorithm. Full HB represents the runtime required for full Hilbert basis computation using Normaliz, and the projected runtime is computed from the probing step of Section 1.2.